ONLINE STORE · VEHICLE WORKFLOW GUIDE

Part 3

From Blender to the browser: the web GLB

The third branch of the shared model. A web model must be small, must carry its animations as named clips, and must tell the viewer which parts are clickable. This part is the recipe, with the export settings and the budgets.

In this part

1. Where this branch leaves the shared scene
2. Budgets: how big a web model may be
3. The poly diet and baking
4. Textures for the web (PBR, like TF3)
5. Hotspots: tagging what is clickable
6. Animations as named clips (NLA strips)
7. Export settings, by hand and with the add-on
8. Checking the result before it goes online
9. Checklist

Export mechanics were verified by exporting a synthetic locomotive with the add-on and loading it in the demo viewer in a headless browser (**demo**). Size budgets are practical guidance, not browser limits.

1 · Where this branch leaves the shared scene

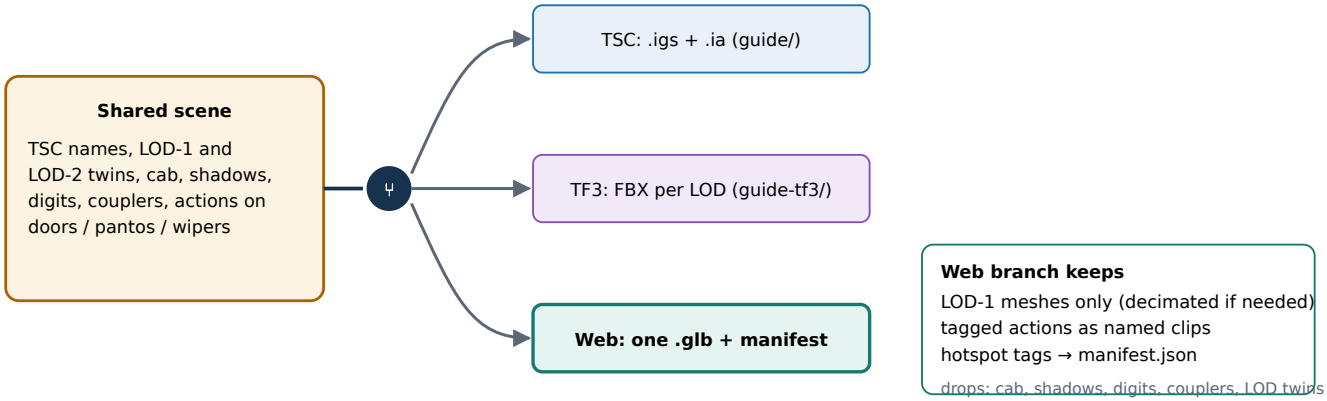


Figure 3.1 · Three exports from one scene. The web branch is the smallest: no cab, no LODs (one is enough at web scale), no game-specific parts. The add-on (Part 7) does the dropping and tagging on a temporary basis so the scene stays the TSC master.

2 · Budgets: how big a web model may be

Measure	Target	Why
Triangles	≤ 60,000 (phones) · ≤ 150,000 (desktop only)	Loads in a second; orbit stays smooth on a laptop GPU. A game-grade loco can be 500 k+.
File size (.glb)	≤ 8 MB, ideally 2–4 MB	Loads before the visitor scrolls away; textures dominate the size.
Textures	≤ 2048 px per sheet, 2–4 sheets, JPEG for colour, PNG only where alpha is needed	A 4 k PNG alone is 20 MB+.
Materials	≤ 6	Each is a separate draw; also keeps the glTF simple.
Animations	3–8 short clips, 0.5–2 s	One per hotspot; keep them crisp.
Nodes	≤ 60	Only parts that move or are clicked need to stay separate; merge the rest.

The demo model is 816 triangles and 0.09 MB because it is primitives; a real loco optimised as below typically lands at 30–60 k triangles and 3–6 MB. demo

3 · The poly diet and baking

1. **Duplicate the LOD-1 model into a “web” collection** (or let the add-on decimate static parts on the fly with a triangle budget). Keep the master untouched.
2. **Delete what the camera never sees:** cab interior, underframe detail below the solebar, bolts, interior walls of closed boxes.
3. **Decimate static parts** (Decimate modifier, Collapse 0.3–0.6) but *not* hotspot parts: doors, pantograph arms, wipers keep their silhouettes.
4. **Merge static parts per material** (body + roof + underframe) so the node count drops; moving parts stay separate objects.
5. **Bake** the high-poly detail onto the low-poly duplicate: normal map, ambient occlusion, and the colour with baked AO if the game textures had none. Cycles ▸ Bake, Selected to Active, cage on.
6. **Apply scale and rotation** (as for both games); origins stay on hinges.

> **Reuse the TF3 textures**
>
> The TF3 branch already produced albedo, normal and metal/gloss/AO sheets. glTF wants almost the same set (base colour, normal, metallic-roughness with AO), so the web branch costs you no new texture work: roughness = 1 − gloss, which the packer script can produce (Part 7).

4 · Textures for the web

glTF slot	Blender Principled input	Notes
baseColorTexture	Base Color	sRGB; JPEG unless alpha is needed.
normalTexture	Normal Map node → Normal	OpenGL convention (Blender's), PNG or JPEG.
metallicRoughnessTexture	Metallic (B) and Roughness (G) via Separate Color	Blender's exporter packs these for you when the inputs are driven by image channels.
occlusionTexture	(glTF Material Output node, Occlusion socket)	Optional; or bake AO into base colour.
emissiveTexture	Emission	Lamps, cab glow.
Alpha / glass	Alpha, Blend Mode = Alpha Blend	Windows; the add-on sets blend mode for materials tagged `web_glass` .

Resize sheets to 2048 or 1024 before export (Image ▸ Resize) and set the exporter's image format to **Automatic** so colour sheets go out as JPEG.

5 · Hotspots: tagging what is clickable

Object: 1_0500_door_l1 (Custom Properties)

```
web_anim = "open_door_l1"
web_label = "Door L1"
```

has an action: closed at frame 0, open at 24

Object: 1_0128_panto01_base

```
web_hotspot = "raise_panto_front"
```

no action itself; clicking it plays the arms' clip

sample_electric.manifest.json (written by the add-on)

```
"hotspots": [
  { "node": "1_0500_door_l1",
    "label": "Door L1", "animations": ["open_door_l1"] },
  { "node": "1_0128_panto01_base",
    "label": "Pantograph front",
    "animations": ["raise_panto_front"] }, … ]
"animations": ["open_door_l1", …, "raise_panto_front", "wipers
"triangles": 816
```

Figure 3.2 · Tag in Blender, manifest out. `web_anim` on an animated object makes it a hotspot that plays its own clip; `web_hotspot` on any object (a door frame, a pantograph base) makes it click-through to a named clip. Labels are inherited per animation, so the two pantograph arms and the base all read “Pantograph front”. **demo**

- Choose 3–8 hotspots per vehicle: the things the product page talks about (doors, pantograph/trolley pole, cab door, headlight, a signature mechanism).
- Make the clickable mesh big enough to hit on a phone; tag the frame or base when the moving part is thin (the pantograph arms are 12 cm wide, the base is 1.6 m).
- The viewer walks up parents on a click, so tagging a parent covers all its children.

6 · Animations as named clips

glTF stores animations as named clips; the viewer plays a clip by name. Blender's exporter can name clips from **NLA tracks**, and the add-on pushes each tagged action into a track/strip named after `web_anim` (the same push the TF3 branch makes, so one scene serves both). Objects sharing a clip name (two pantograph arms, two wipers) merge into one glTF animation. **demo**

- Key the *open* state only; the viewer plays the clip backwards on the second click (“close”).
- Frame 0 = rest; 12–30 frames at 24 fps; linear or ease-out keys; no scale.
- Looping things (wipers, fans) are still exported once; the viewer plays once per click. Make the clip a full cycle so it returns to rest.

7 · Export settings

By hand (File ▸ Export ▸ glTF 2.0)

- Format: **glTF Binary (.glb)**
- Include: Selected Objects (the web collection)
- Transform: **+Y Up** ✓
- Mesh: Apply Modifiers ✓, UVs ✓, Normals ✓
- Material: Export, Images: Automatic
- Animation ✓ · **Group by NLA Track** ✓ · Sampling ✓ (force) · Limit to playback range ✗
- Compression: off for the first test (Draco needs a decoder in the page; add later if size matters)

With the add-on (Part 7)

- TSC sidebar ▸ **Web viewer** ▸ Output folder = your site's `models/`
- Model name, triangle budget (60,000), max texture size (2048)
- **Export web GLB + manifest**
- Reads: LOD-1 objects, skips TSC-only parts, decimates static parts if over budget, pushes tagged actions to NLA strips, exports with the settings on the left, writes the manifest.
- Reports triangle count, file size, oversized textures.

8 · Checking the result before it goes online

1. Copy the `.glb` and `.manifest.json` into `store-demo/models/` and add the manifest path to `MODEL_POOL` in `index.html`.
2. Start the local server (Part 5 §1) and open the page. The status line shows triangles and animation count.
3. Click every hotspot button and every part; hover labels should match. Watch for: a door opening the wrong way (key direction), a part jumping at the start (rest pose not at frame 0), black materials (missing base colour), see-through faces (flipped normals: Mesh ▸ Normals ▸ Recalculate Outside).
4. Resize the window to phone width; orbit with touch if you can.
5. Check the file size in the browser's Network tab (F12) and the frame rate while orbiting.

9 · Checklist

✓	Item
☐	Under the triangle, size and texture budgets
☐	Static parts merged per material; moving/clickable parts separate with origins on hinges
☐	3–8 hotspots tagged with labels; thin parts tagged via a base/frame
☐	Each clip named, rest at frame 0, open state keyed; shared names merged
☐	Glass tagged; emissive set for lamps
☐	Exported +Y up, .glb, NLA tracks; manifest written
☐	Loaded and clicked in the local viewer; phone width checked